

✓ Stage 3 — Data Extraction

Objective

The objective of this stage is to extract useful information from the transaction dataset for subsequent validation, aggregation, analysis, and visualization.

The timestamp field is converted into a datetime format and useful temporal attributes are extracted to make time-based analysis easier.

Relevant transaction attributes are also selected to create a focused dataset for the subsequent stages.

Operations Performed

1. Loaded the transaction dataset.
2. Converted the timestamp field into datetime format.
3. Extracted date, month, year, week number, day of month, and day name.
4. Selected the relevant transaction attributes.
5. Verified the extracted dataset structure.
6. Displayed the extracted fields and sample records.

Output

A focused extraction dataset is created containing the transaction attributes required for analysis together with useful temporal attributes derived from the timestamp.

```
# =====
# STAGE 3 - CONTAINER 1
# SOURCE DATASET
# =====

import pandas as pd
import numpy as np

df = pd.read_csv("/content/Dataset(1).csv")

print("SOURCE DATASET")
print("=" * 70)
print(f"Number of records : {df.shape[0]:,}")
print(f"Number of columns : {df.shape[1]}")

print("\nDataset columns:")
for i, column in enumerate(df.columns, start=1):
    print(f"{i}. {column}")
```

```
SOURCE DATASET
=====
Number of records : 250,000
Number of columns : 27

Dataset columns:
1. Transaction ID
2. time_stamp
3. Transaction_Type
4. merchant category
5. amount_INR
6. Transaction Status
7. sender_agegroup
8. receiver_age_group
9. Sender State
10. sender bank
11. Receiver_Bank
12. device type
13. Network Type
14. Fraud Flag
15. hour of day
16. Day Of Week
17. is weekend
18. battery level %
19. App Version
20. device storage free gb
```

- 21. screen brightness %
- 22. Device Language
- 23. GPS accuracy m
- 24. nearby wifi count
- 25. session duration sec
- 26. notifications last hour
- 27. merchant distance km

Timestamp Extraction

The timestamp field contains the date and time information for each transaction. Converting it into datetime format allows individual temporal components to be extracted and analysed separately.

The extracted temporal attributes are:

- Date
- Month
- Year
- Week number
- Day of month
- Day name

These attributes make it possible to study transaction behaviour across different periods and days.

```
# =====
# STAGE 3 - CONTAINER 2
# TEMPORAL ATTRIBUTE EXTRACTION
# =====

df["time_stamp"] = pd.to_datetime(
    df["time_stamp"],
    errors="coerce"
)

df["date"] = df["time_stamp"].dt.date
df["month"] = df["time_stamp"].dt.month
df["year"] = df["time_stamp"].dt.year
df["week_number"] = df["time_stamp"].dt.isocalendar().week.astype("Int64")
df["day_of_month"] = df["time_stamp"].dt.day
df["day_name"] = df["time_stamp"].dt.day_name()

print("TEMPORAL ATTRIBUTES EXTRACTED")
print("=" * 70)

temporal_columns = [
    "time_stamp",
    "date",
    "month",
    "year",
    "week_number",
    "day_of_month",
    "day_name"
]

display(df[temporal_columns].head())
```

TEMPORAL ATTRIBUTES EXTRACTED

=====

	time_stamp	date	month	year	week_number	day_of_month	day_name
0	2024-10-08 15:17:00	2024-10-08	10.0	2024.0	41	8.0	Tuesday
1	NaT	NaT	NaN	NaN	<NA>	NaN	NaN
2	2024-04-02 13:27:00	2024-04-02	4.0	2024.0	14	2.0	Tuesday
3	2024-01-07 10:09:00	2024-01-07	1.0	2024.0	1	7.0	Sunday
4	2024-01-23 19:04:00	2024-01-23	1.0	2024.0	4	23.0	Tuesday

Transaction Attribute Extraction

The transaction fields required for the analysis are selected and combined with the temporal attributes derived from the timestamp. This creates a focused extraction dataset that can be passed to the next stages without repeatedly selecting the same fields.

```
# =====
# STAGE 3 - CONTAINER 3
# TRANSACTION ATTRIBUTE EXTRACTION
# =====

extraction_columns = [
    " Transaction ID ",
    "time_stamp",
    "Transaction_Type",
    " merchant category ",
    "amount_INR",
    "Transaction Status",
    "sender_agegroup",
    "receiver_age_group ",
    "Sender State",
    "sender bank",
    "Receiver_Bank",
    "device type",
    "Network Type ",
    "Fraud Flag",
    "hour of day",
    "Day Of Week",
    "is weekend"
]

extracted_df = df[extraction_columns].copy()

extracted_df["date"] = df["date"]
extracted_df["month"] = df["month"]
extracted_df["year"] = df["year"]
extracted_df["week_number"] = df["week_number"]
extracted_df["day_of_month"] = df["day_of_month"]
extracted_df["day_name"] = df["day_name"]

print("EXTRACTED DATASET")
print("=" * 70)
print(f"Number of records : {extracted_df.shape[0]:,}")
print(f"Number of columns : {extracted_df.shape[1]}")
```

```
EXTRACTED DATASET
=====
Number of records : 250,000
Number of columns : 23
```

```
# =====
# STAGE 3 - CONTAINER 4
# EXTRACTED DATASET PROFILE
# =====

extracted_profile = pd.DataFrame({
    "Column": extracted_df.columns,
    "Data Type": extracted_df.dtypes.astype(str).values,
    "Non-Null Values": extracted_df.notna().sum().values,
    "Unique Values": extracted_df.nunique(dropna=True).values
})

display(extracted_profile)
```

	Column	Data Type	Non-Null Values	Unique Values
0	Transaction ID	object	250000	250000
1	time_stamp	datetime64[ns]	226106	170831
2	Transaction_Type	object	244939	55
3	merchant category	object	244936	91
4	amount_INR	object	245364	19230
5	Transaction Status	object	244959	35
6	sender_agegroup	object	245437	58
7	receiver_age_group	object	245428	57
8	Sender State	object	245428	91
9	sender bank	object	245434	81
10	Receiver_Bank	object	245443	83
11	device type	object	245415	36
12	Network Type	object	245422	46
13	Fraud Flag	object	245856	6
14	hour of day	object	245955	28
15	Day Of Week	object	245921	11
16	is weekend	object	245923	6
17	date	object	226106	365
18	month	float64	226106	12
19	year	float64	226106	1
20	week_number	Int64	226106	52
21	day_of_month	float64	226106	31
22	day_name	object	226106	7

Extracted Data Preview

The first few records are displayed to verify that the selected transaction attributes and the derived temporal information are aligned with the original transaction records.

```
# =====
# STAGE 3 - CONTAINER 5
# SAMPLE EXTRACTED TRANSACTIONS
# =====

print("SAMPLE EXTRACTED TRANSACTIONS")
print("=" * 70)

display(extracted_df.head())
```

SAMPLE EXTRACTED TRANSACTIONS

```
# =====
# STAGE 3 - CONTAINER 6
# EXTRACTION SUMMARY
# =====

print("EXTRACTION SUMMARY")
print("=" * 70)

print(f"Original records : {df.shape[0]:,}")
print(f"Extracted records: {extracted_df.shape[0]:,}")
print(f"Original attributes selected: {len(extraction_columns)}")
print(f"Derived temporal attributes : 6")

print("\nDerived temporal attributes:")
for i, column in enumerate(
    ["date", "month", "year", "week_number", "day_of_month", "day_name"],
    start=1
):
    print(f"{i}. {column}")
```

```
EXTRACTION SUMMARY
=====
Original records : 250,000
Extracted records: 250,000
Original attributes selected: 17
Derived temporal attributes : 6

Derived temporal attributes:
1. date
2. month
3. year
4. week_number
5. day_of_month
6. day_name
```

Stage 3 — Conclusion

The transaction timestamp was successfully converted into a datetime format and used to extract date, month, year, week number, day of month, and day name.

The relevant transaction attributes were also extracted into a focused analysis dataset. These extracted fields provide the required foundation for later validation, aggregation, time-based comparisons, and visualization.

The extracted information will be used in the following stages to examine transaction behaviour across transaction categories, time periods, fraud status, and other relevant dimensions.